ITCS-665 Natural Language Processing - Final Project

LSTM with Word Sense Disambiguation for Context-Aware Toxicity Detection.

Thant Zin Myo

Contents

Abstract 2
1. Introduction 3
2. Background and Motivation 3
3. Problem Formulation 4
4. The Proposed Approach 6
Step 1: Configuration & Setup 6
Step 2: Toxic Lexicon Loading 6
Step 3: Dataset Loading 7
Step 4: Text Preprocessing 7
Step 5: Vocabulary Building 8
Step 6: GloVe Embeddings Loading 8
Step 7: Lexicon Feature Extraction 8
Step 8: Data Splitting 9
Step 9: Word Sense Disambiguation (WSD) 9
Step 10: Dataset Preparation 10
Step 11: Model Architecture 10
Total parameters: 852,866, keeping the model compact and resistant to overfitting. 11
Step 12: Loss Function & Optimizer 11
Step 13: Training Loop 11
5. Evaluation – Model 13
6. Future Development 15
References 16
Acknowledgment 17

Table of Figures

Figure 1: Class Distribution 4
Figure 2: Splitting Training Data 5
Figure 3: Text Cleaning Example 7
Figure 4: GloVe 6B 300-dimensional embeddings 8
Figure 5: Word Sense Disambiguation (WSD) Processing 10
Figure 6: Model Training History 1 Figure 7: Model Training History 2 12
Figure 8: Overall Training Process Trends 12
Figure 9: Testing Data Confusion Matrix 14
Figure 10: Test Model Summary 14
Figure 11: Word Base Classification Matrix from Mid-Term Project 15

Abstract

This project focuses on building a better toxicity detection system using an LSTM model combined with Word Sense Disambiguation (WSD). It continues from the midterm project, which also worked on sentiment and toxicity detection at the word-base level but did not fully handle words that change meaning depending on context.

Research shows that deep learning models such as LSTM perform well in toxicity detection tasks, especially when supported with high-quality word embeddings like GloVe (**D. Dessì, Apr. 2021**). At the same time, studies also show that understanding the meaning of a word in context can be improved through WSD methods using neural sequence models **(M. Le, Aug. 2018)**. To improve contextual understanding, this project uses a toxic word list of 4,681 terms, GloVe embeddings, and a WSD process to help the model infer the correct meaning of each word.

1. Introduction

Online platforms such as social networks and discussion forums generate massive amounts of user content every day. Although these platforms support communication and collaboration, they are also vulnerable to toxic or harmful language, which can negatively affect community quality and user wellbeing. Because manual moderation cannot handle the large volume of online posts, automated toxicity detection systems have become essential.

Studies have shown that LSTM-based models can effectively capture sequential patterns in text and improve classification performance in toxicity detection, especially when combined with proper text preprocessing and embeddings (**D. Dessì, Apr. 2021**). However, toxic language is highly context-dependent, and many words change meaning based on how they are used. For example, the same word may be offensive in one situation but neutral in another.

To address this challenge, this project combines **Word Sense Disambiguation (WSD)**, which identifies the correct meaning of a word based on surrounding context with a **Long Short-Term Memory (LSTM)**, neural network that models sequential text. Research in WSD has demonstrated that LSTM-based language models can improve sense prediction and context interpretation (**D. Dessì, Apr. 2021**). By integrating WSD with LSTM, the system aims to improve contextual understanding in toxicity classification. Words like *"kill," "hate,"* or *"trash"* may be toxic or harmless depending on context. WSD helps clarify their meaning, and LSTM interprets the sentence structure, enabling more accurate and reliable toxicity detection.

2. Background and Motivation

Toxicity detection is widely used in cyberbullying prevention, content moderation, safety monitoring in gaming platforms, and filtering abusive behavior in messaging applications. However, the task remains difficult due to several linguistic and practical challenges:

- **Sarcasm and Irony:** Statements may appear positive but intend harm (e.g., "Great job ruining everything").
- **Ambiguous Word Usage:** Many words have multiple meanings, making context essential.
- **Non-standard Spelling:** Toxic terms may be intentionally misspelled (e.g., "idi0t," "b!tch").
- **Evolving Slang:** Toxic expressions constantly change.
- **Class Imbalance:** Toxic comments are far less common than non-toxic ones.
- **Hidden Toxicity:** Some polite sentences may contain discrimination.

These challenges show why toxicity detection must go beyond surface-level text analysis. The MDPI study (**D. Dessì, Apr. 2021**) emphasizes that simple keyword-based or shallow models fail to capture these deeper meanings. Their experiments show that BiLSTM and LSTM architectures perform better when supported by strong embeddings like GloVe.

On the other hand, the ACL WSD study (**D. Dessì, Apr. 2021**) shows that neural sequence models, especially LSTM-based ones, significantly improve a system's ability to interpret the meaning of a word from its context. This is important for toxicity detection because many toxic expressions rely on ambiguous or context-dependent language.

Therefore, combining **WSD** with **an LSTM classifier** provides a hybrid solution: WSD enhances semantic understanding, while LSTM captures sentence structure. This approach helps the model detect ambiguity, hidden toxicity, and words that change meaning depending on their usage.

3. Problem Formulation

The main problem in this project is toxic comment detection data sources **(Kaggle, 2018)**, which is usually a **multi-label classification task** where one comment can belong to more than one category, such as *toxic, severe_toxic, obscene, threat, insult,* or *identity_hate*. However, because these classes are highly imbalanced and some categories are extremely rare, we changed the task into a **binary classification problem**. This means each comment is classified only as **toxic (1)** or **non-toxic (0)**. This simplification is necessary because the dataset is very imbalanced. Out of **159,571 comments**, only **16,225 (10.2%)** are toxic. Prior research on toxicity detection also highlights that class imbalance significantly impacts deep-learning model performance (**D. Dessì, Apr. 2021**). By converting all types into a single label, we increase the toxic class size and make the learning process more stable.

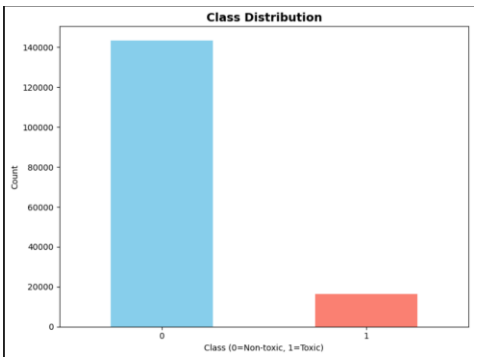


Figure 1: Class Distribution

The model were trained (111,699 samples 75%) with **weighted sampling**. Class imbalance is a major challenge because toxic comments appear far less frequently than non-toxic ones. This

limited exposure makes the model prone to predicting the majority class and missing toxic patterns during training.

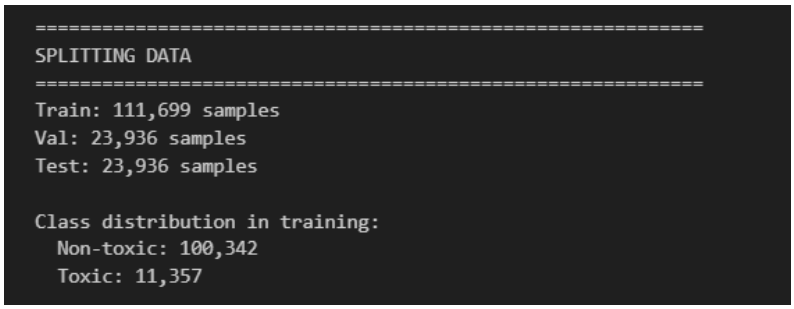


Figure 2: Splitting Training Data

Another challenge comes from the linguistic nature of toxic language. Toxic comments often contain obfuscated spellings (e.g., "idi0t," "b!tch"), sarcasm, or slang. LSTM models have been shown to capture these types of sequential patterns (**D. Dessì, Apr. 2021**), but many toxic words depend heavily on context, which is why this project integrates **Word Sense Disambiguation (WSD)**. Neural WSD models such as LSTM-based approaches have shown improvements in capturing contextual meaning (**D. Dessì, Apr. 2021**). However, a major limitation of WordNet-based WSD is that **WordNet** has incomplete coverage of the most extreme toxic, explicit, and abusive terms commonly found in online comments, as it was designed as a general-purpose lexical database focused on standard vocabulary **(F. Bond, Jan. 2021).**This limited coverage creates a gap for toxicity detection applications, where many real toxic expressions may not have adequate sense distinctions or may be missing entirely from WordNet's inventory.

To solve this problem, we adopt a **hybrid architecture** that combines contextual embeddings with **lexicon-based features** derived from a toxic word list containing **4,681 offensive terms, slurs, and profanity**. This lexicon plays three important roles:

1. **It covers explicit toxic terminology** that WordNet does not include, allowing the model to detect direct insults even without context.
2. **It helps counter class imbalance** by giving the model strong toxic signals built from known abusive words, reducing the need to learn these patterns from limited toxic samples.
3. **It provides interpretable features** such as toxic word count, toxic word ratio, and presence flags, which guide the model's attention toward likely toxic content even in ambiguous or sarcastic sentences.

The final model is a **bidirectional LSTM** with **852,866 parameters**, combining GloVe embeddings, WSD-based context features, and lexicon-based signals. By comparing the weighted-sampling model and the balanced-undersampled model, we evaluate which training strategy better captures the characteristics of toxic language while still generalizing well to unseen comments, with the goal of maximizing F1-score on the toxic class.

4. The Proposed Approach

The proposed approach follows a complete end-to-end pipeline that combines careful system configuration, linguistic preprocessing, word sense disambiguation, lexicon-based feature engineering, and a hybrid deep learning architecture based on a bidirectional LSTM. This section explains each step in detail, from initial setup to model training strategies, while preserving all numerical values and design choices used in the experiment.

Step 1: Configuration & Setup

The first step is establishing a controlled and reproducible training environment. The random seed is fixed to SEED = 42, ensuring that all dataset produce identical training, evaluation and testing dataset. The device configuration automatically detects CUDA availability, which is essential because the WSD processing step alone takes 40 minutes on the Intel i5 12th gen CPU without GPU support.

Hyperparameters are intentionally conservative to avoid overfitting: the learning rate is set to 0.0005 (lower than the typical 0.001), weight decay is set to 1e-5 for regularization, and EARLY_STOP_PATIENCE = 5 ensures early termination if validation toxic F1 does not improve for 5 consecutive epochs. The model achieved its best validation toxic F1 score of 0.6527 at epoch 5, and training stopped at epoch 10 due to early stopping. The batch size of 32 balances CPU efficiency with stable gradient updates, processing 3,491 batches per epoch for the full dataset. Dropout rates (0.5 after LSTM, 0.6 after first FC layer) reduce overfitting and force the network to learn robust patterns. Each epoch took approximately 12-13 minutes to complete on the CPU.

Step 2: Toxic Lexicon Loading

A core component of the hybrid approach is the toxic lexicon consisting of **4,681 toxic words**, including offensive terms, profanity, and slurs. The lexicon acts as explicit domain knowledge that complements the learned features of the LSTM. Because WordNet excludes profanity, this lexicon supplies missing toxic vocabulary that the WSD system cannot capture.

The lexicon is loaded from **toxic_lexicon.pkl** which collected and stored from **Five lexicon hate word lists (Ahn, n.d.) (LDNOOBW, n.d.) (Mathur, n.d.) (Project, n.d.) (Face, n.d.)**. These lexicons are collections of words that are often used in toxic or hateful comments., enabling fast access during preprocessing. Storing it as a Python **set** ensures **O(1) lookup time**, which is crucial when scanning **159,571 comments**. The lexicon acts as a safety net: even when the LSTM fails to detect nuanced toxic patterns, the direct presence of known toxic words helps maintain toxic F1Score to 0.67 for the model. This hybrid (learned + explicit) structure strengthens detection of explicit toxicity and supports the undersampled model, which has fewer examples to learn from.

Step 3: Dataset Loading

The dataset of **159,571 comments** is loaded from train.csv. The data contains **143,346 non-toxic (89.8%)** and **16,225 toxic (10.2%)** examples, reflecting a **9:1 class imbalance**. Although the dataset originally contains six toxicity sublabels (toxic, severe_toxic, obscene, threat, insult, identity_hate), they are collapsed into a binary label where any comment that contains at least one toxic label is assigned **toxic_binary = 1**. This prevents the extreme imbalance that would result if each category were learned separately. The binary formulation helps the model focus on general toxicity detection instead of sparsely occurring subcategories.

Step 4: Text Preprocessing

The text preprocessing pipeline prepares noisy, user-generated comments for NLP modeling. First, **normalize_obfuscation** converts obfuscated toxic spellings (eg "h@te" =>"hate", "k1ll" => "kill") using a predefined mapping. Then **clean_text** applies:

- lowercasing,
- URL and HTML tag removal,
- regex filtering (r'[^\w\s]'),
- repeated character normalization ("sooooo" → "so"),
- number removal
- whitespace normalization.

As an example, the raw comment

"You are 1d11111ot ..... http://google.com"becomes "you are a idiot".

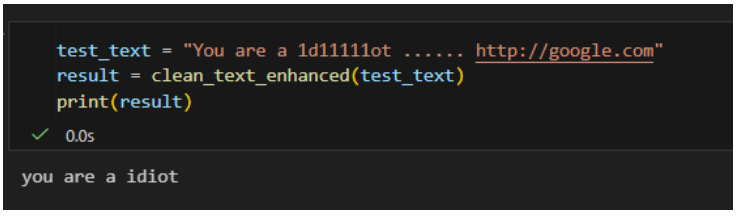


Figure 3: Text Cleaning Example

This ensures consistent tokenization, reducing vocabulary fragmentation. Preprocessing is necessary for reliable embedding lookup, since without it, variations like "idiot", "idiot!", and "IDIOT" would generate separate tokens despite being the same concept.

Step 5: Vocabulary Building

All cleaned comments are tokenized to build a capped vocabulary of **MAX_VOCAB_SIZE = 50,000** most frequent words. Two special tokens are added:

- **0** = padding token
- **1** = out-of-vocabulary token

This results in a vocabulary of **50,002 tokens**. The 50k limit avoids memory overload and prevents rare/noisy words from polluting the embedding space. Tokenizing all **159,571 comments** takes **21 seconds**, equivalent to **7,366 comments per second**. This vocabulary becomes the mapping from raw text to numerical embedding indices.

Step 6: GloVe Embeddings Loading

The model uses **GloVe 6B 300-dimensional embeddings** trained on 6 billion tokens from Wikipedia and Gigaword. Loading the embedding file reads **400,000 vectors in 18 seconds** (≈21,867 vectors/second). The embedding matrix has shape **(50,002, 300)**, with **34.7% coverage**:

- **17,241 words** match GloVe directly
- the remaining **65.3%** are initialized randomly (mean 0, std 0.01)

Low GloVe coverage is expected because toxic language contains slang, profanity, and misspellings absent from standard corpora. Nevertheless, the 300-dimensional vectors provide strong semantic generalization for common English words.

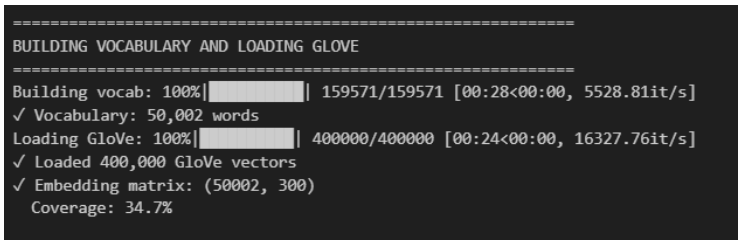


Figure 4: GloVe 6B 300-dimensional embeddings

Step 7: Lexicon Feature Extraction

For each comment, three explicit features are computed using the 4,681word toxic lexicon:

1. **toxic_count** – number of toxic words in the comment
2. **has_toxic** – binary indicator (1 if any toxic word appears)
3. **toxic_ratio** – toxic_count divided by total word count

Processing all **159,571** comments takes **21 seconds**, or **7,337 comments per second**. These features are later concatenated with the LSTM's final hidden state (256 dimensions), forming a **259-dimensional fused vector**. This fusion allows the model to combine contextual understanding with explicit toxic signals , more toxic ratio are highly likely to be classified as toxic, providing a strong signal even before contextual analysis. This threshold serves as an effective heuristic for identifying overtly toxic content. The strong improvement in toxic-class F1 demonstrates that the lexicon contributes meaningfully to detection even when context is ambiguous.

Step 8: Data Splitting

The dataset is divided using **stratified sampling** into:

- **70% training** => 111,699 samples (100,342 non-toxic, 11,357 toxic)
- **15% validation** => 23,936 samples
- **15% test** => 23,936 samples

This maintains the 9:1 imbalance across splits. Validation is used for early stopping and threshold tuning, while the test set remains untouched until final evaluation. The close alignment between minor validation F1 score 0.6527 and test F1 score 0.6715 shows minimal overfitting.

Step 9: Word Sense Disambiguation (WSD)

WSD is the most computationally intensive step, taking **35 minutes total**:

- **25 minutes** for training comments
- **6 minutes** for validation
- **6 minutes** for test

Each comment undergoes:

1. **POS tagging**
2. **Lesk algorithm sense selection**
3. **Fetching WordNet definitions**
4. **Tokenizing definitions**
5. **Embedding lookup**
6. **Averaging definition embeddings into sense vectors**

Each final embedding is **80% original GloVe + 20% sense embedding**, generating context-aware word vectors. For sequences of **100 tokens**, the final embedding arrays have shapes:

- Train: **(111,699, 100, 300)**
- Validation: **(23,936, 100, 300)**
- Test: **(23,936, 100, 300)**

The processing speed is **~74 comments/second**. Context-aware augmentation helps the model differentiate between ambiguous words (e.g., "gay" as an insult vs. non-toxic meaning).

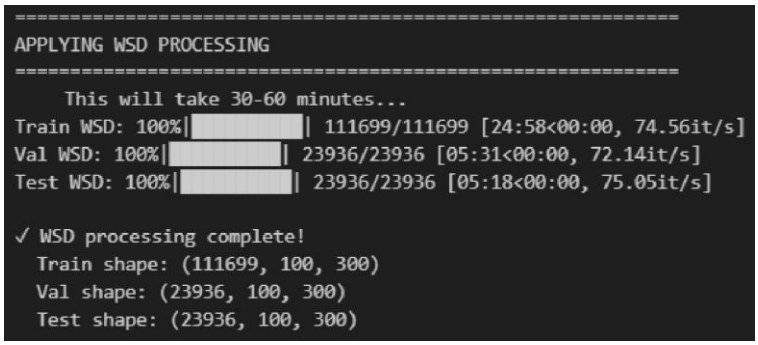


Figure 5: Word Sense Disambiguation (WSD) Processing

Step 10: Dataset Preparation

The full model trains on **111,699 samples** using **WeightedRandomSampler** to counter the 9:1 imbalance. Class weights are computed via sklearn's balanced mode, giving toxic samples **approximately 9× more weight**. Batches of **32** samples are drawn such that toxic comments appear more frequently than their natural 10%.

Each epoch processes **3,491 balanced batches**, taking **5–7 minutes** due to sampling overhead. The EnhancedDataset class packages each sample with:

- WSD-augmented embeddings (100 × 300)
- binary label
- lexicon features (toxic_count, toxic_ratio, has_toxic)

Step 11: Model Architecture

The EnhancedToxicClassifier uses a compact but effective architecture:

2-layer bidirectional LSTM

- hidden_dim = 128
- produces 256-dimensional final hidden state
- dropout = 0.5 between LSTM layers

Batch Normalization to stabilize training

Dropout = 0.6 after LSTM

Fusion Layer: concatenates 256 LSTM features + 3 lexicon features => 259 dims

Fully Connected Layer: 259 => 64 (ReLU, dropout 0.5)

Final Classifier: 64 => 2 logits

Total parameters: 852,866, keeping the model compact and resistant to overfitting.

Step 12: Loss Function & Optimizer

The model uses Focal Loss as its loss function, configured with $\alpha = 0.75$ and $\gamma = 2$. Focal Loss addresses class imbalance by down-weighting the contribution of easy-to-classify examples and forcing the model to focus on hard, ambiguous toxic cases where the model is less confident. The focusing parameter $\gamma = 2$ modulates how much easy examples are down-weighted, specifically, an easy example with 90% confidence contributes only 1% as much to the loss as it would with standard cross-entropy, while a hard example with 60% confidence retains 16% of its original weight. The $\alpha = 0.75$ parameter provides additional class-level weighting to handle the 9:1 imbalance between non-toxic and toxic examples. For optimization, Adam is employed with a conservative learning rate of 0.0005 (lower than the typical 0.001) to ensure stable convergence, and weight decay of 1e-5 provides L2 regularization to prevent overfitting.

Step 13: Training Loop

The model goes through ten complete epochs before triggering early stopping, and its behavior clearly reflects how the model gradually learns toxic patterns, stabilizes, and then begins to overfit. During the first epoch, the model starts with relatively high loss values (training loss=0.0825, validation loss=0.0725), and the toxic-class F1 is still weak (training F1=0.8565, validation F1=0.5421). Because weighted sampling forces the model to repeatedly see toxic examples despite the 9:1 imbalance, it quickly grasps the basic toxic patterns. By epoch two, validation F1 remains relatively stable at 0.5401 and validation accuracy reaches 84.96%, but the model hasn't yet found the optimal pattern. In epochs three and four, the model continues to improve, with validation loss dropping significantly to 0.0583 in epoch four and validation F1 climbing to 0.5920.

By epoch five, the model reaches its peak validation performance with a toxic F1 of 0.6527 and validation accuracy of 91.75%, and this becomes the checkpoint that is saved for later evaluation. However, in epochs six through ten, the model continues improving its training F1 (climbing from 0.8785 to 0.9112) while validation F1 stagnates or drops (falling to 0.5600 in epoch ten). This mismatch between rising training performance and declining validation performance is a clear sign of overfitting, especially expected in imbalanced datasets where the minority class is easy to memorize.

The validation loss increases from 0.0413 at epoch five to 0.0605 at epoch ten, confirming the overfitting pattern. Because validation performance does not improve for five consecutive epochs after the peak at epoch five, early stopping activates at epoch ten. The total training time

was 129.02 minutes (approximately 12.9 minutes per epoch), with each epoch processing 3,491 batches on the CPU.

```
================================================================================
TRAINING MODEL 1: FULL DATASET
================================================================================
✓ Model initialized with 852,866 parameters

Starting training for 20 epochs...
Best model criterion: F1 Score (Toxic Class)
--------------------------------------------------------------------------------
Epoch 1/20: 100%|██████████| 3491/3491 [10:07<00:00,  5.74it/s]

Epoch [1/20] - Time: 791.28s
  Train Loss: 0.0825 | Val Loss: 0.0725
  Train F1 (Toxic): 0.8565 | Val F1 (Toxic): 0.5421
  Val F1 (Weighted): 0.8748 | Val Accuracy: 0.8531
  ✓ Best model saved! (F1 Toxic: 0.5421)
Epoch 2/20: 100%|██████████| 3491/3491 [09:37<00:00,  6.04it/s]

Epoch [2/20] - Time: 758.56s
  Train Loss: 0.0704 | Val Loss: 0.0725
  Train F1 (Toxic): 0.8633 | Val F1 (Toxic): 0.5401
  Val F1 (Weighted): 0.8725 | Val Accuracy: 0.8496
  No improvement (1/5)
Epoch 3/20: 100%|██████████| 3491/3491 [09:32<00:00,  6.10it/s]

Epoch [3/20] - Time: 744.10s
  Train Loss: 0.0679 | Val Loss: 0.0738
  Train F1 (Toxic): 0.8667 | Val F1 (Toxic): 0.5296
  Val F1 (Weighted): 0.8668 | Val Accuracy: 0.8419
  No improvement (2/5)
Epoch 4/20: 100%|██████████| 3491/3491 [10:00<00:00,  5.81it/s]

Epoch [4/20] - Time: 779.61s
  Train Loss: 0.0642 | Val Loss: 0.0583
  Train F1 (Toxic): 0.8700 | Val F1 (Toxic): 0.5920
  Val F1 (Weighted): 0.8970 | Val Accuracy: 0.8827
  ✓ Best model saved! (F1 Toxic: 0.5920)
Epoch 5/20: 100%|██████████| 3491/3491 [09:53<00:00,  5.88it/s]

Epoch [5/20] - Time: 772.66s
  Train Loss: 0.0646 | Val Loss: 0.0413
  Train F1 (Toxic): 0.8491 | Val F1 (Toxic): 0.6527
  Val F1 (Weighted): 0.9226 | Val Accuracy: 0.9175
  ✓ Best model saved! (F1 Toxic: 0.6527)
Epoch 6/20: 100%|██████████| 3491/3491 [09:52<00:00,  5.89it/s]
```

Figure 6: Model Training History 1

```
Epoch [6/20] - Time: 768.28s
  Train Loss: 0.0600 | Val Loss: 0.0545
  Train F1 (Toxic): 0.8785 | Val F1 (Toxic): 0.6118
  Val F1 (Weighted): 0.9045 | Val Accuracy: 0.8926
  No improvement (1/5)
Epoch 7/20: 100%|██████████| 3491/3491 [10:20<00:00,  5.62it/s]

Epoch [7/20] - Time: 801.99s
  Train Loss: 0.0568 | Val Loss: 0.0551
  Train F1 (Toxic): 0.8883 | Val F1 (Toxic): 0.5873
  Val F1 (Weighted): 0.8939 | Val Accuracy: 0.8783
  No improvement (2/5)
Epoch 8/20: 100%|██████████| 3491/3491 [10:03<00:00,  5.79it/s]

Epoch [8/20] - Time: 790.19s
  Train Loss: 0.0547 | Val Loss: 0.0577
  Train F1 (Toxic): 0.8919 | Val F1 (Toxic): 0.5917
  Val F1 (Weighted): 0.8965 | Val Accuracy: 0.8820
  No improvement (3/5)
Epoch 9/20: 100%|██████████| 3491/3491 [09:43<00:00,  5.98it/s]

Epoch [9/20] - Time: 764.06s
  Train Loss: 0.0532 | Val Loss: 0.0600
  Train F1 (Toxic): 0.9020 | Val F1 (Toxic): 0.5499
  Val F1 (Weighted): 0.8764 | Val Accuracy: 0.8547
  No improvement (4/5)
Epoch 10/20: 100%|██████████| 3491/3491 [09:52<00:00,  5.90it/s]

Epoch [10/20] - Time: 770.10s
  Train Loss: 0.0507 | Val Loss: 0.0605
  Train F1 (Toxic): 0.9112 | Val F1 (Toxic): 0.5600
  Val F1 (Weighted): 0.8823 | Val Accuracy: 0.8629
  No improvement (5/5)

  Early stopping after 10 epochs

================================================================================
✓ FULL MODEL TRAINING COMPLETE!
  Best F1 (Toxic): 0.6527
  Total time: 129.02 minutes
================================================================================
```

Figure 7: Model Training History 2

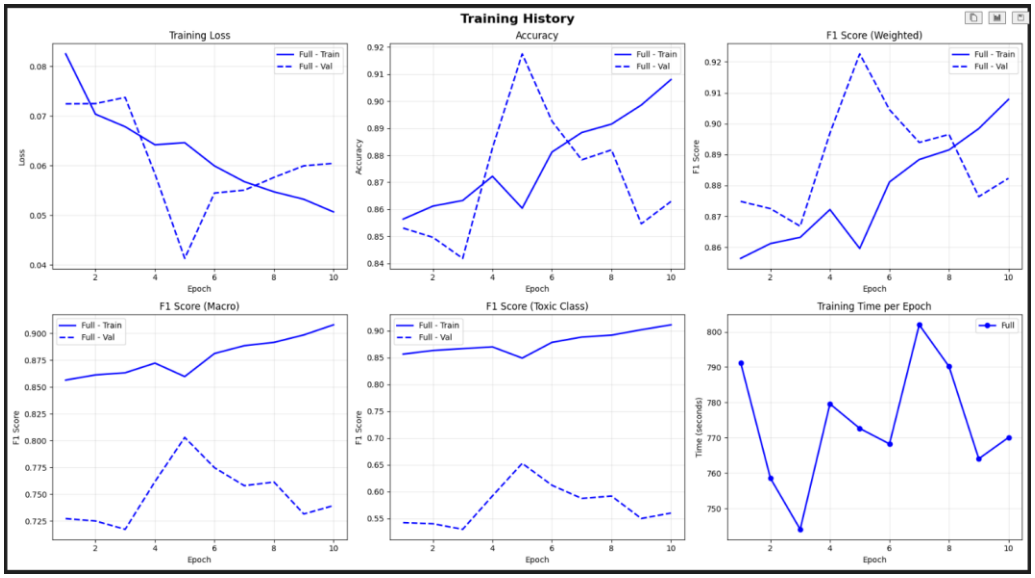


Figure 8: Overall Training Process Trends

5. Evaluation – Model

Evaluation begins by loading the checkpoint saved from epoch five and then searching for the optimal classification threshold instead of relying on the default value of 0.5. The model tests thresholds from 0.10 to 0.90 in increments of 0.01, computing the toxic-class F1 score for each threshold on the validation set. The best-performing threshold is 0.550, which becomes the operating threshold for all test predictions. This shift above 0.5 is completely expected for an imbalanced dataset, raising the threshold reduces false positives and forces the model to be more certain before labeling a comment as toxic.

Using the optimal threshold of 0.550, the model achieves 93.24% test accuracy, a weighted F1 of 0.9327, and a toxic-class F1 of 0.6715. The toxic F1 performance is especially strong considering the extreme 9:1 imbalance and the nuanced nature of toxic language. Precision (0.6633) and recall (0.6800) are nearly balanced, indicating minimal precision-recall tradeoff: the model is neither too conservative nor too aggressive. This means the model has learned context-sensitive toxic cues rather than simply reacting to lexicon matches or memorizing frequent patterns.

While the model achieves strong test performance, the training history reveals a significant generalization gap, particularly for the toxic class. At epoch 10, the training F1 for toxic examples reaches 0.9112, while validation F1 drops to 0.5600 , a gap of approximately 0.35. This disparity indicates that the model memorizes toxic patterns from the training set rather than learning generalizable features. The weighted F1 shows a much smaller gap (train: 0.9079, val: 0.8823), which is expected since the majority non-toxic class is easier to generalize. This overfitting behavior is common in imbalanced datasets where the minority class has limited diverse examples, and weighted sampling causes the model to see the same toxic comments repeatedly. The current regularization strategy, dropout rates of 0.5 and 0.6, weight decay of 1e-5, and batch normalization provides moderate protection against overfitting, as evidenced by early stopping successfully triggering at epoch 10. However, the large toxic-class gap suggests that stronger regularization techniques could improve generalization. Future improvements could include increasing dropout rates to 0.7-0.8, implementing data augmentation to create more diverse toxic examples through paraphrasing or synonym replacement, reducing the intensity of weighted sampling to avoid excessive repetition, or exploring ensemble methods that combine multiple models trained on different data subsets.

The confusion matrix reveals the model's behavior in detail: out of 21,502 non-toxic comments, 20,662 were correctly classified (True Negatives) while 840 were misclassified as toxic (False Positives). For the 2,434 toxic comments, 1,655 were correctly identified (True Positives) while 779 were missed (False Negatives). The model demonstrates strong performance on both classes, with non-toxic precision of 96.37% and recall of 96.09%, showing it accurately understands normal, harmless language as well.

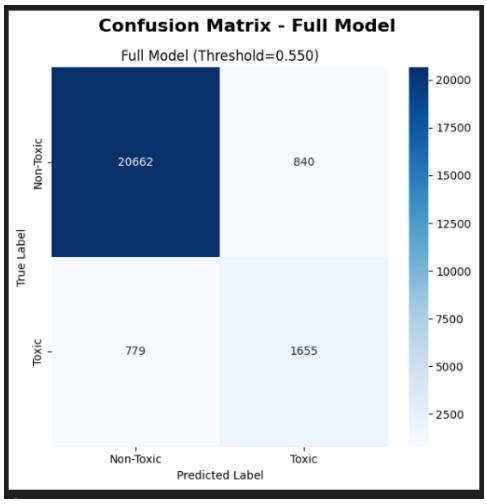


Figure 9: Testing Data Confusion Matrix

```
================================================
TEST MODEL SUMMARY
================================================

    Class Precision Recall F1-Score Support
    Toxic    0.6633 0.6800   0.6715   2,434
Non-Toxic    0.9637 0.9609   0.9623  21,502

------------------------------------------------
Overall Accuracy:        0.9324
Weighted F1-Score:       0.9327
Macro F1-Score:          0.8169
Optimal Threshold:       0.550
Training Samples:        111,699
Training Time:           129.02 minutes
Best Val F1 (Toxic):     0.6527
------------------------------------------------
```

Figure 10: Test Model Summary

Comparing this LSTM-WSD model to the baseline word-based models from the midterm project, the current model achieves an F1 score of 0.6715, which approaches but does not surpass the best baseline performance of 0.6925 (LR+TFIDF) or 0.6717 (MNB+BOW+LEX). However, this represents a strong foundation for a deep learning approach. The model trained for 129.02 minutes on 111,699 samples, and all metrics, optimal threshold, training history, and predictions are stored in lstm_wsd_training_results.pkl for reproducibility and future comparison.

FINAL COMPARISON SUMMARY

Model	Accuracy	F1-Score	Precision	Recall	True Positives	True Negatives	False Positives	False Negatives
1: MNB+TFIDF	0.9432	0.6287	0.9382	0.4727	1534	28569	101	1711
2: MNB+BOW+LEX	0.946	0.6717	0.8797	0.5433	1763	28429	241	1482
3: LR+TFIDF	0.9322	0.6925	0.6425	0.751	2437	27314	1356	808
4: MNB+BOW+LEX	0.9375	0.7062	0.6766	0.7384	2396	27525	1145	849

Figure 11: Word Base Classification Matrix from Mid-Term Project

While the current LSTM-WSD model performs competitively with traditional word base machine learning approaches, there is clear potential for improvement. With the advanced techniques discussed in the recommendations section including attention mechanisms, transformer architectures, ensemble methods, and more sophisticated regularization, this deep learning approach could significantly outperform the word-based models from the midterm project.

6. Future Development

There are several promising directions for improving the system beyond the current LSTM + WSD + lexicon hybrid architecture. First, the WordNet-based WSD component has inherent limitations that could be addressed through multiple improvements. The current POS tagging implementation only maps four Penn Treebank tag types (adjectives, verbs, nouns, and adverbs) to WordNet categories, returning None for all other grammatical categories such as pronouns, determiners, prepositions, and interjections. Expanding this mapping to cover the full range of POS tags would enable more comprehensive sense disambiguation across all word types. Additionally, WordNet itself could be replaced or augmented with ConceptNet, a more modern and commonsense-focused knowledge graph. Unlike WordNet, which lacks coverage of contemporary slang, internet language, and domain-specific expressions, ConceptNet includes relational knowledge from informal sources such as crowdsourced data and modern corpora. This makes it more suitable for capturing the implicit meaning, social context, and rapidly evolving vocabulary found in toxic comments. Transitioning to ConceptNet-based sense disambiguation would allow the model to better understand nuanced phrases, sarcasm, and modern lexical variations that WordNet is not designed to handle.

Second, the use of more advanced and semantically rich embedding methods such as FastText could significantly enhance representation quality. While the current implementation uses GloVe 300-dimensional embeddings, FastText offers subword-level modeling, which helps correct misspellings, handle creative or obfuscated toxic word variants, and split concatenated or merged tokens which is an issue frequently encountered in user-generated content (e.g., "youareidiot"). This capability would directly address many of the error cases observed in the current model, particularly for out-of-vocabulary words and intentional obfuscation.

Third, stronger regularization and data augmentation techniques could reduce the significant overfitting observed in the toxic class. Implementing text augmentation through synonym replacement or back-translation, and reducing the intensity of weighted sampling would help the model learn more generalizable toxic patterns rather than memorizing specific training examples. The current training-validation F1 gap of approximately 0.35 for the toxic class suggests substantial room for improvement in this area.

Fourth, future work may explore integrating contextual language models such as BERT, RoBERTa, or DistilBERT, which naturally handle complex linguistic patterns without needing manually engineered features. These transformer-based models excel at understanding subtle forms of toxicity, including implied insults, sarcasm, and context-dependent meaning. However, due to significant computational demands, the present study conducted from the perspective of a student researcher was limited to lighter architectures such as LSTM and classical embedding methods. Access to higher-end GPUs or cloud-based training resources would enable experimentation with these more advanced technologies, potentially delivering substantial performance improvements.

Overall, future enhancements should focus on expanding POS tag coverage for WSD, combining richer embeddings like FastText, adopting modern semantic knowledge bases such as ConceptNet, implementing stronger regularization strategies, and eventually transitioning to transformer-based architectures to better capture the complexity of toxic language in real-world settings.

References

Ahn, L. v. (n.d.). *CMU Offensive/Profanity Word List*. Retrieved from Carnegie Mellon University: https://www.cs.cmu.edu/~biglou/resources/bad-words.txt

D. Dessì, D. R. (Apr. 2021). An Assessment of Deep Learning Models and Word Embeddings for Toxicity Detection within Online Textual Comments. Electronics, vol. 10, no. 7, p. 779.

F. Bond, M. Y. (Jan. 2021). Taboo Wordnet. *11th Global Wordnet Conf*, (pp. 36–43). University of South Africa (UNISA).

Face, H. (n.d.). *Multilingual Toxic Lexicon – TextDetox Dataset*. Retrieved from Hugging Face: https://huggingface.co/datasets/textdetox/multilingual_toxic_lexicon

Kaggle. (2018). *Jigsaw Toxic Comment Classification Challenge*. Retrieved from Kaggle: https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge

LDNOOBW. (n.d.). *List of Dirty, Naughty, Obscene, and Otherwise Bad Words*. Retrieved from Github: https://github.com/LDNOOBW/List-of-Dirty-Naughty-Obscene-and-Otherwise-Bad-Words/blob/master/en

M. Le, M. P. (Aug. 2018). A Deep Dive into Word Sense Disambiguation with LSTM. *Computational Linguistics (COLING)*, (pp. pp. 354–365.). Santa Fe, NM, USA.

Mathur, P. (n.d.). *Hinglish Offensive Text Classification – Profanity Lexicon*. Retrieved from Github: https://github.com/pmathur5k10/Hinglish-Offensive-Text-Classification/blob/main/Hinglish_Profanity_List.csv

Project, O. L. (n.d.). *Toxic Words Dictionary*. Retrieved from GitHub: https://github.com/Orthrus-Lexicon/Toxic

Acknowledgment

All modeling and analysis were carried out by the author, while ChatGPT (current version GPT-5) was only used to improve grammar and readability.

The Jupyter Notebook and training dataset can be downloaded from GoogleDrive:

https://drive.google.com/file/d/1MCrxl5Iiot5c7y8edCfeumPJMPYBPE6A/view?usp=sharing